# Агенты и их оркестрация

Андрей Крисанов, 2026

# Агент
## определение

**Агент** – это *автономный* (в разумных пределах) программный субъект, который:

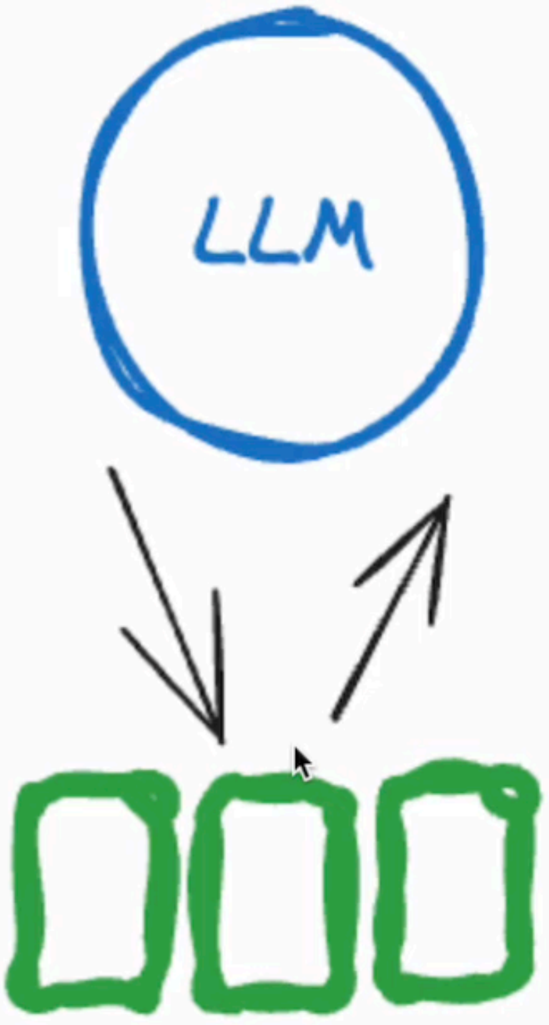
- получает задачу или цель
- планирует шаги
- выбирает действия и выполняет их через tools (API, БД, файловая система, sandbox, поиск, очереди и пр.)
- поддерживает состояние (state/memory) между шагами
- использует обратную связь от среды выполнения чтобы корректировать план
- останавливается по критерию завершения (done/failed/cancelled)
- возвращает результат

Agent = LLM + state + tools + loop + policies

LLM – это не агент, а компонент принятия решений внутри агента

# Single Agent

- Слишком много инструментов в наборе → плохие решения насчет вызова следующего инструмента
  💡 5-10 tools макс.
- Контекст растет быстро для одного агента → сложно его поддерживать и понимать → деградация latency / качества
- Потребность в нескольких специализированных действиях в продукте (планирование, исследование, вычисления и пр.)

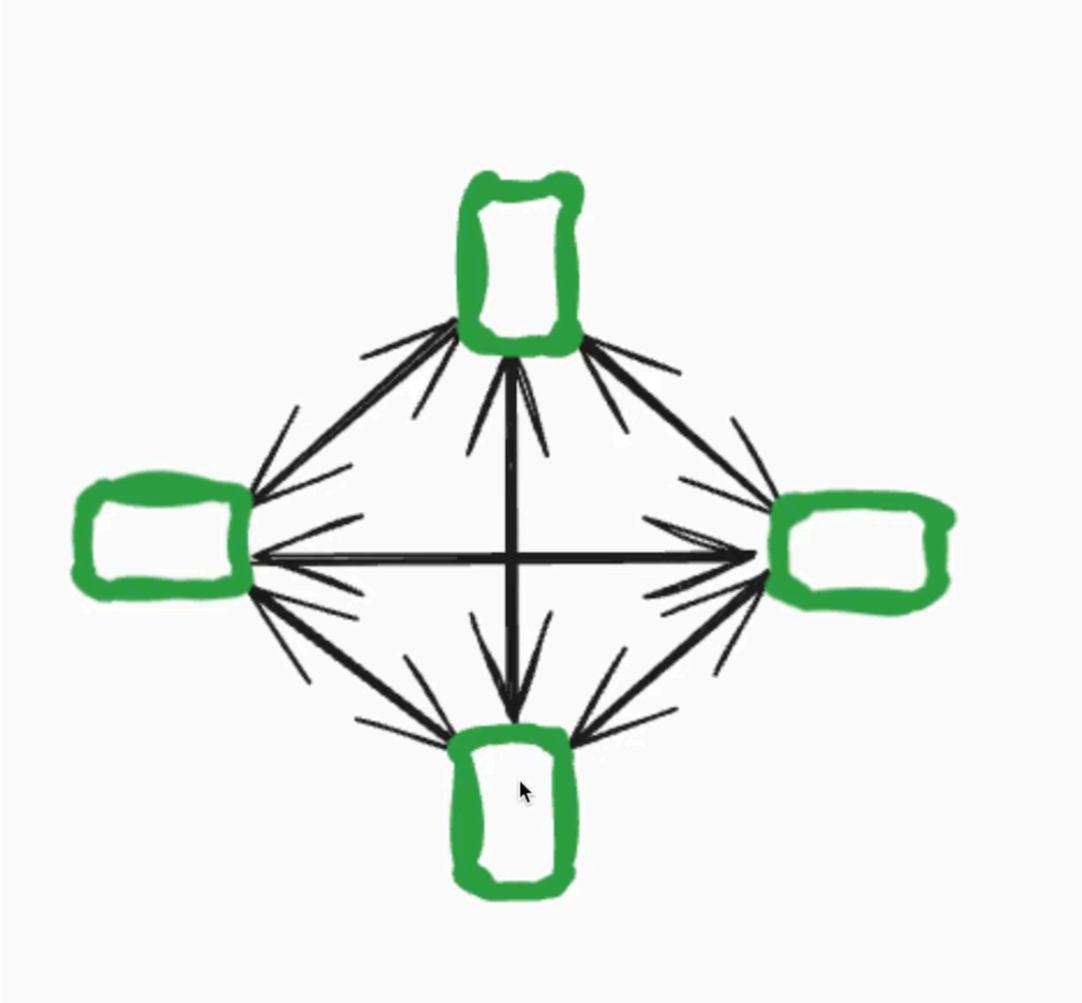


# Плюсы мульти-агентных систем

- **Модальность:** Разделение агентов упрощает разработку, тестирование и поддержку агентской системы.
- **Специализация:** Возможность создания эксперта в конкретной доменной области → улучшение производительности системы.
- **Контроль:** Можем явно контролировать как агенты коммуницируют вместо того, чтобы полагаться на вызов тулов моделью / одним агентом.

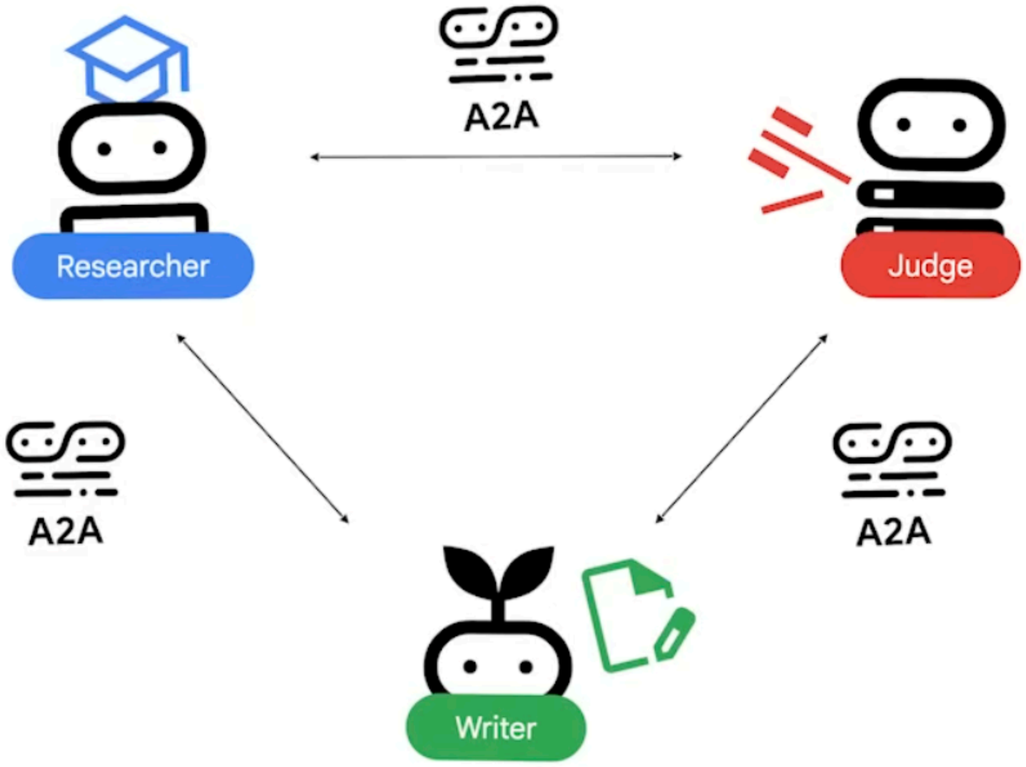
# Сеть

- У каждого агента свой набор инструментов
- Взаимодействуют друг с другом решая кто действует следующим
- Фреймворки: Swarm, CrewAI
- Сложно контролировать поведение → ненадежность и дороговизна
- Не рекомендуется использовать в продакшене



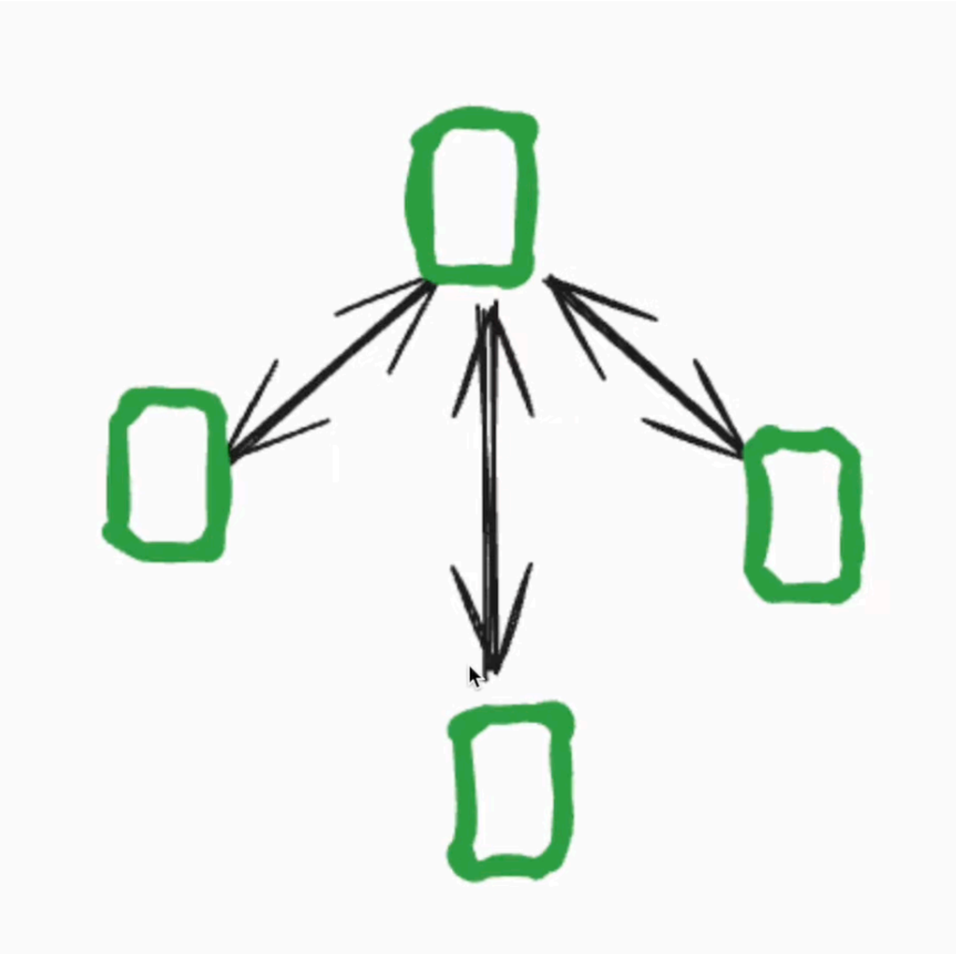
# A2A

https://a2a-protocol.org/latest/



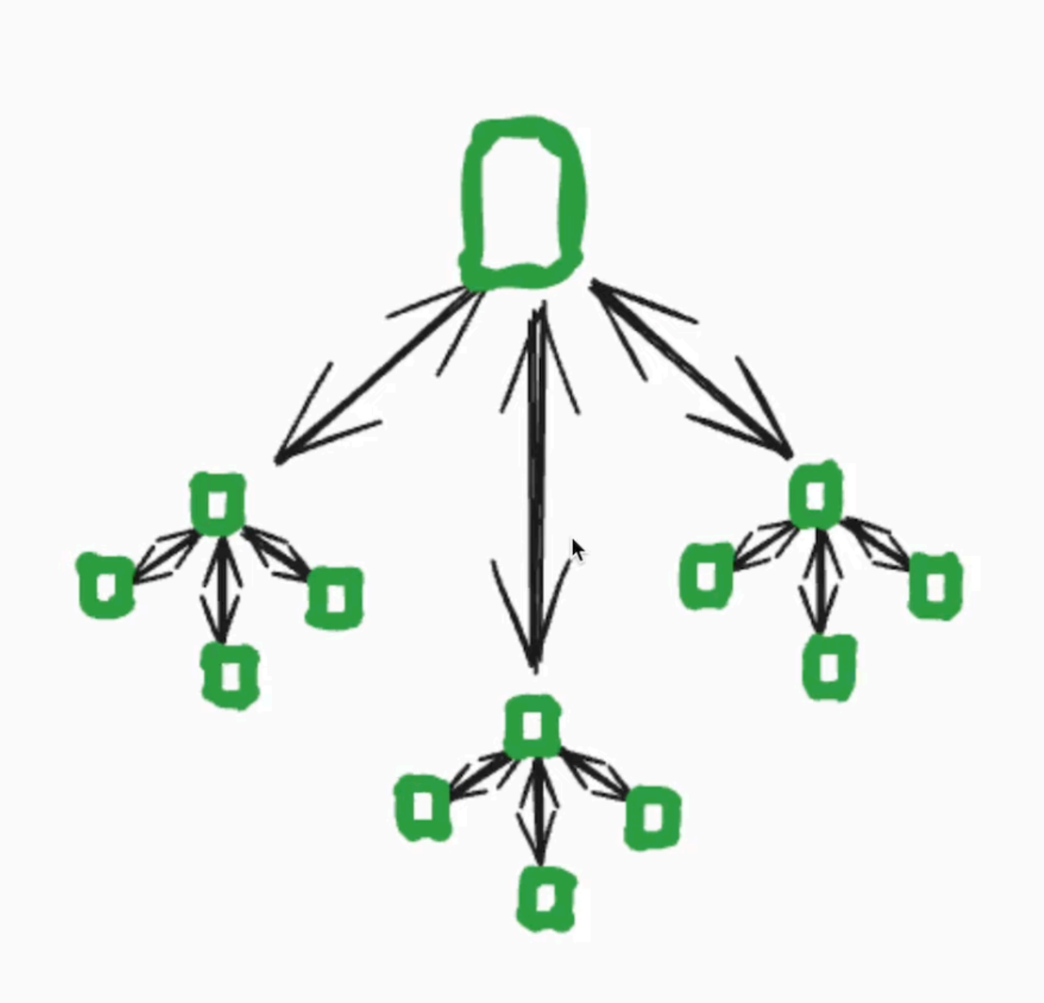
# Супервайзер

- Агент, *единственная* задача которого – передавать запросы другими агентам
- Под-агенты фокусируются на своей задаче и не думать кого вызывать следующим
- Супервайзер решает кого вызывать следующим → есть общее состояние
- Упрощенная версия – LLM вызывает агентов как инструменты и коммуникация через параметры тулов



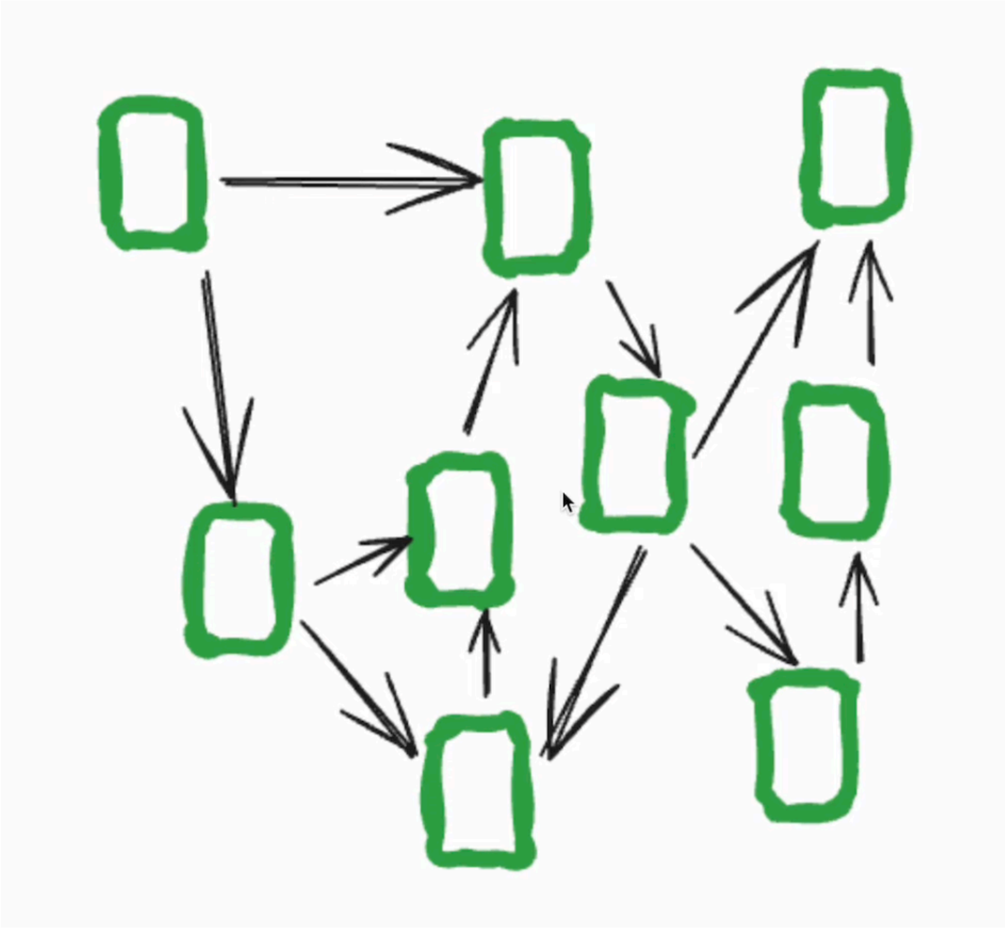
# Иерархия

- Иерархия супервайзер-агентов
- Хорошо масштабируется когда в системе много агентов и появляется необходимость их группировки (напр., по домену)



# В реальных проектах

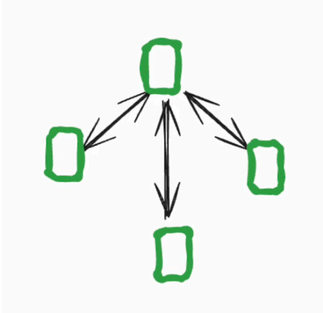
- Часто своя собственная архитектура под конкретные экспертные домены



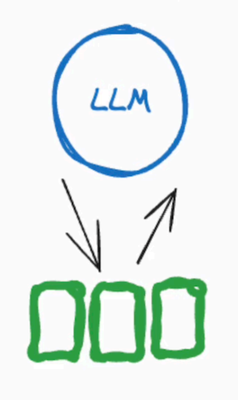
# Обмен состоянием между агентами

## Что видит Agent 2 вызываемый Agent 1

- *Общее состояние* графа {“messages”: “...”, “artifacts”: “”} → меняем состояние после ответа

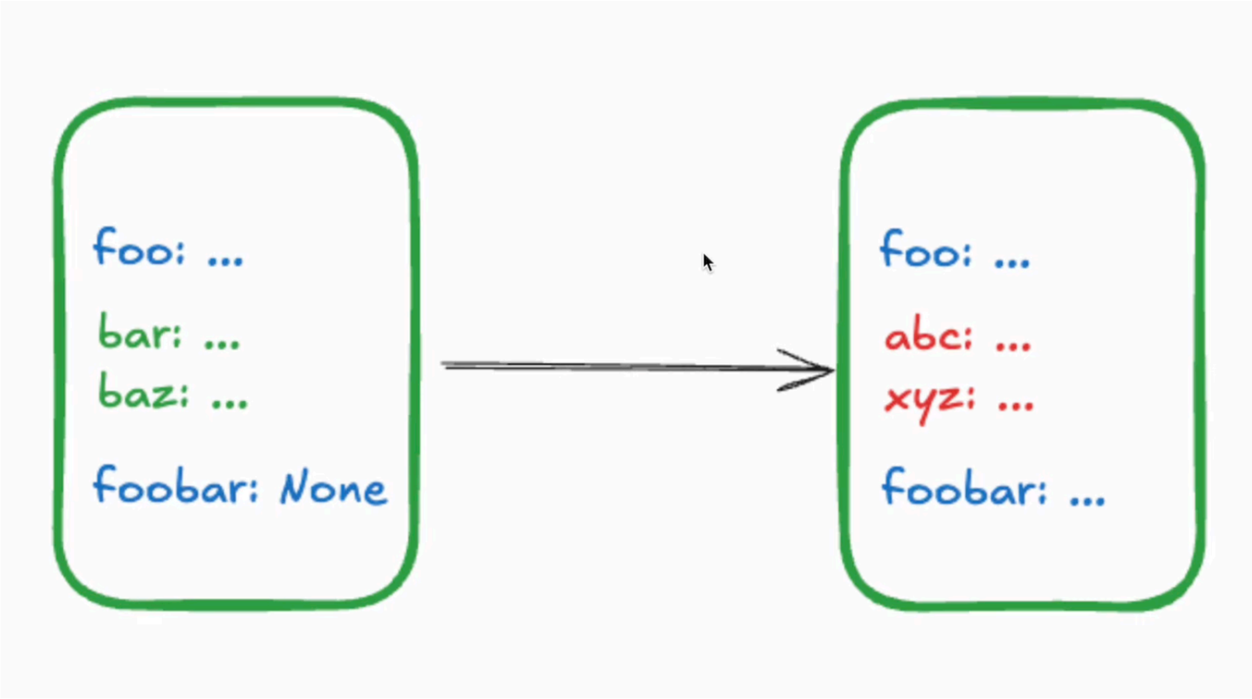


- *Аргументы* с которыми Agent 2 был вызван как инструмент (tool call) {“arg1”: “...”, “arg2”: “...”} → возвращаем результат как ответ инструмента



# Коммуникация агентов
## с разным состоянием

- Общие ключи посредством которых выполняется коммуникация
- Специфичное для агента состояние хранится в различающихся ключах
- Agent 2 возвращает foobar: value и Agent 1 может использовать его



# Обмен состоянием между агентами

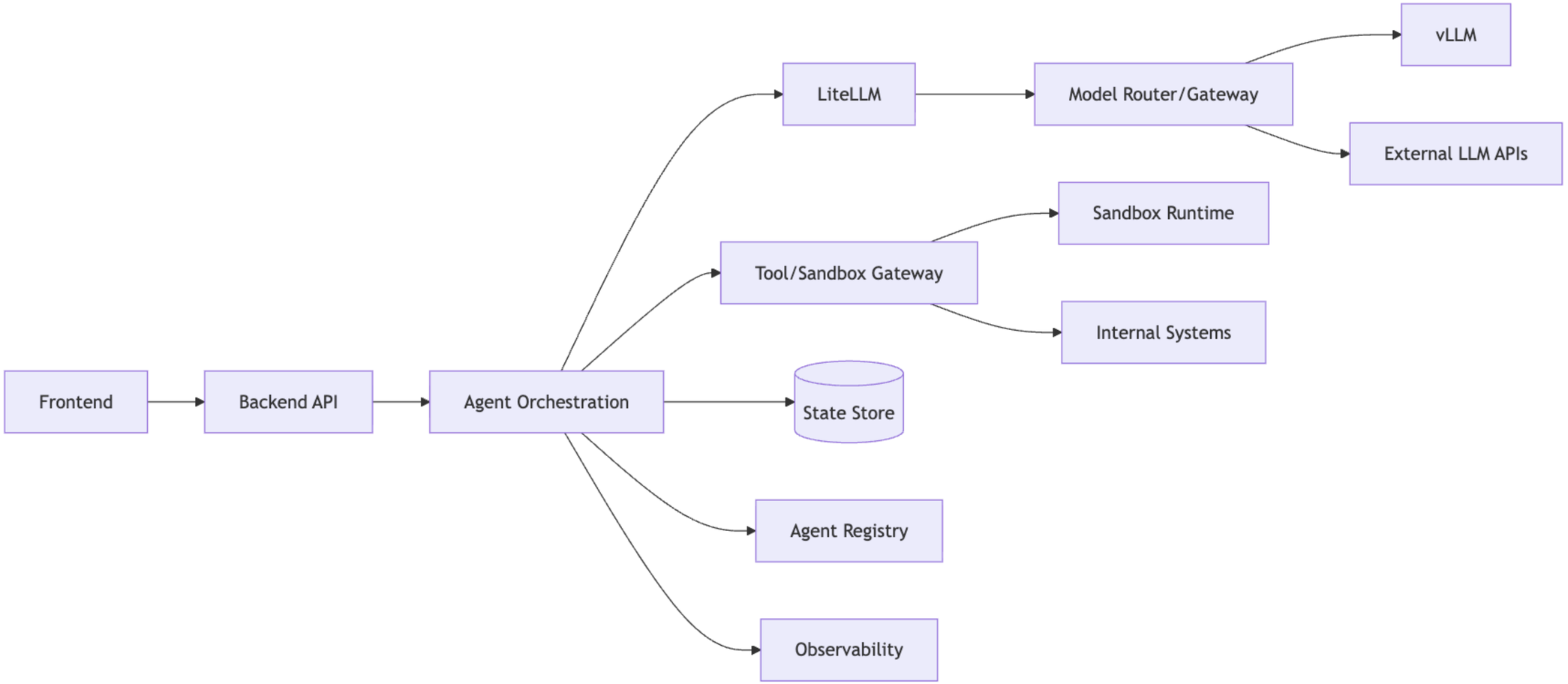
## Что видит Agent 1 когда Agent 2 возвращает ответ

- Финальный ответ Agent 2 + внутренний процесс рассуждения, включая результаты вызова его инструментов → растущий контекст
- Только финальный ответ Agent 2

# Требования к мульти-агентской оркестрации

- **Реестр агентов:** оркестратор знает всех агентов и их параметры
- **Контроль доступа и безопасность:** управление агентами и доступом к ресурсам
- **Модель выполнения:** контролируемое выполнение (TTL, retries, failures) с отслеживанием состояния – агентов можно останавливать, продолжать их выполнение; контракты вызова инструментов
- **Память и артефакты:** безопасная модель состояния и хранилище метаданных и документов
- **Управление нагрузкой на инференс:** политики использования моделей, лимиты, квоты и приоритеты, защита от зацикливаний
- **Наблюдаемость:** observability для отладки и мониторинга

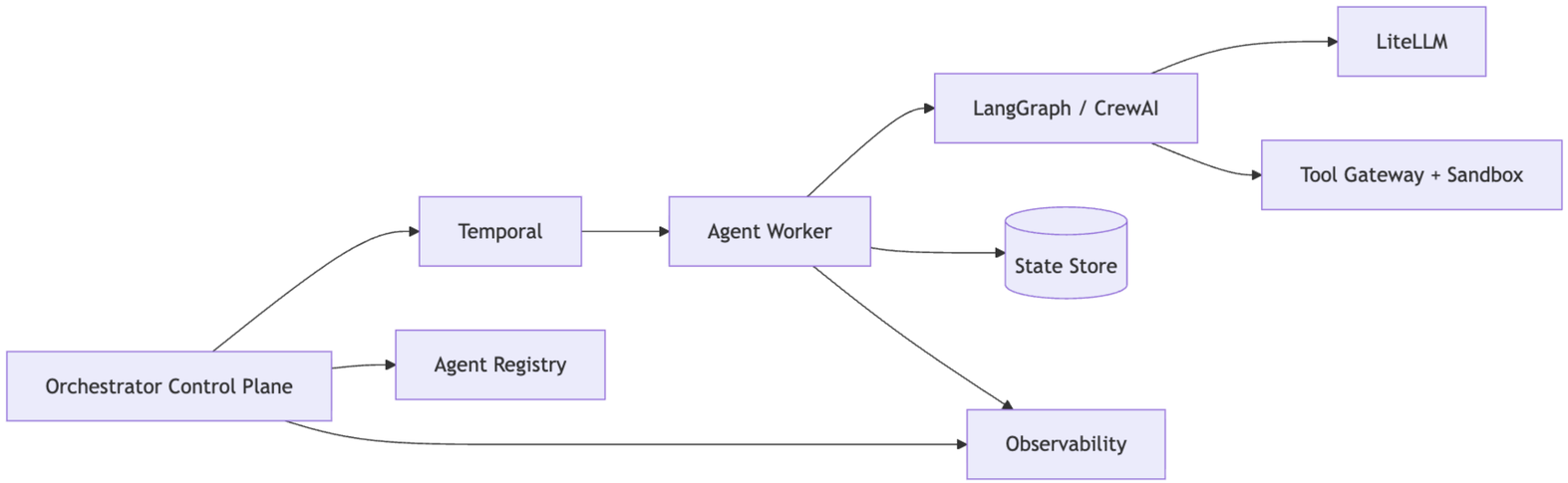
# Общая архитектурная диаграмма



# Возможная архитектура

## С применением OSS решений

- **Control Plane:** знает всех агентов, управляет доступом
- **Temporal:** оркестрирует агентский workflow
- **LangGraph/CrewAI:** среда выполнения



# Как это работает

- UI / API инициирует run (запуск)
- Control Plane решает можно ли запускать задачу, проверяет лимиты, права и модели → стартует отказоустойчивый Temporal Workflow
- Workflow выполняет шаги агента через Activity
- Внутри Activities запускаются внутреннюю логику агента, реализованную через LangGraph / CrewAI
- Все внешние вызовы идут через наши адаптеры → LiteLLM или Tool Gateway

# Missing Pieces

## Как развивать платформу дальше

- **Human-in-the-Loop:** явный механизм Approval Gate – состояние, в котором агент замирает и ждет внешнего сигнала через API или UI для продолжения. В схеме с Temporal это реализуется через Signals.
- **Evals:** без оценки агентов не сможем понять стало ли системе лучше после обновления промпта или смены модели. Нужен механизм запуска golden datasets (тестовых наборов) на графе агентов.
- **Prompt Management:** управление и версионирование промптов → LangFuse (?)
- **Guardrails:** глобальные правила на input/output системы. Примеры: проверка PII, токсичность и пр. перед передачей данных от агента.
- **Semantic Caching:** если агент уже искал информацию по похожему запросу, результат можно взять из кэша, не вызывая дорогой запрос (LiteLLM(?)) → снижает latency и стоимость.

# Каталог моделей
## Дополнительный компонент

- Представляет собой список ролей и разрешенных реализаций
- reasoning_llm: Kimi K2 Thinking, DeepSeek V3.2
- Для каждой роли есть:
  - список разрешенных пользователей и агентов
  - максимальное число токенов / контекст
  - параметры конкурентного доступа
  - цепочка fallback в случае отказа
  - бюджет на стоимость и время отклика